SYSTEM ARCHITECT AUTOMOTIVE

GIÁO TRÌNH CHUYÊN SÂU AUTOSAR LIN

Cơ chế Kéo (Pull) dữ liệu từ Application đến UART Peripheral

Biên soạn: **Phạm Văn Vũ**

HALA Academy

GIÁO TRÌNH CHUYÊN SÂU AUTOSAR LIN STACK

Từ `main()` đến LIN Bus, rồi quay ngược lại đường xác nhận truyền

> Tài liệu này bám sát đúng code demo trong repository hiện tại. Mục tiêu không chỉ là mô tả "có các lớp AUTOSAR", mà là chỉ rõ **dữ liệu đang nằm ở đâu, ai gọi ai, tại sao LIN dùng Pull theo lịch, và cuối cùng chuỗi byte nào thực sự đi ra `USART2`**.

> **Một câu chốt bản chất của demo:** Ứng dụng không đẩy frame LIN xuống bus. Ứng dụng chỉ cập nhật *signal* vào `COM shadow buffer`. Đến đúng slot, `LinIf` mới kéo payload mới nhất qua `PduR -> Com`, đóng gói thành `Lin_PduType`, rồi yêu cầu `Lin` phát ra `USART2`.

Mục lục

1. Mục tiêu học tập và bức tranh toàn cục
2. LIN là gì ở góc nhìn kỹ thuật hệ thống
3. Vì sao AUTOSAR LIN dùng Pull/Time-Triggered
4. Mapping trực tiếp vào code và cấu hình của repo
5. Luồng thực thi đầy đủ từ `main()` đến `USART2`
6. Phân tích LIN frame ở mức byte/bit
7. Timeline thực tế của những vòng lặp đầu tiên
8. Sơ đồ chi tiết đã render sẵn
9. Kiểm chứng bằng log và lab mô phỏng
10. Các điểm dễ nhầm và kết luận kỹ thuật

---

1. Mục tiêu học tập và bức tranh toàn cục

Sau khi học xong tài liệu này, người học cần trả lời được các câu hỏi sau mà không còn mơ hồ:

- Tại sao mạng LIN Master không hoạt động theo kiểu "`Com_SendSignal()` xong là frame lập tức lên bus"?
- Trong code demo, dữ liệu tồn tại lần lượt ở những bộ đệm nào trước khi biến thành byte vật lý?

- Sự khác nhau giữa **Frame ID thô** (`0x10` , `0x11`) và **Protected Identifier - PID** trên wire (`0x50` , `0x11`) là gì?
- Vai trò của `Com` , `PduR` , `LinIf` và `Lin` khác nhau ở chỗ nào?
- Vì sao demo có thêm nhánh `TxConfirmation` ở vòng lặp kế tiếp, dù hàm driver đang gửi theo kiểu blocking?
- `USART1` trong demo dùng để làm gì, và `USART2` mới là ngoại vi LIN thật sự ở đâu?

> **Chuỗi đi đúng của dữ liệu trong demo:**
> `Application update signal -> COM shadow buffer -> LinIf scheduler -> PduR_LinIfTriggerTransmit -> Com_TriggerTransmit -> LinIf local TX buffer -> Lin_SendFrame -> USART2 -> LIN transceiver -> LIN bus`

---

2. LIN là gì ở góc nhìn kỹ thuật hệ thống

2.1 Bản chất của LIN

LIN (Local Interconnect Network) là mạng truyền thông chi phí thấp cho các chức năng thân xe hoặc tải chậm như đèn, gương, kính, khóa cửa, HVAC. So với CAN, LIN đơn giản hơn nhiều nhưng đánh đổi bằng tốc độ thấp hơn và cơ chế truy cập bus kém linh hoạt hơn.

Đặc tính	LIN	Ý nghĩa trong demo
Cấu trúc bus	Master-Slave	`LinIf` đóng vai trò bộ điều phối lịch phát của master.
Cơ chế truy cập bus	Time-triggered theo *Schedule Table*	Không có chuyện ứng dụng tự giành quyền phát frame bất cứ lúc nào.
Tầng vật lý	Single-wire LIN bus thông qua LIN transceiver	`USART2` chỉ là nguồn phát byte nối tiếp; muốn lên bus ô tô thật cần transceiver.
Tốc độ	Thường 19.2 kbps	Code demo cấu hình `USART2` ở `19200` baud.
Khung dữ liệu	Break + Sync + PID + Data + Checksum	`Lin_SendFrame()` là nơi tạo ra chuỗi byte/bit này.

2.2 Cấu trúc một khung LIN hoàn chỉnh

Trường	Kích thước	Ai phát	Ý nghĩa
Break	≥ 13 bit dominant	Master	Đánh dấu bắt đầu frame mới, đánh thức mọi slave đang chờ header.
Sync	1 byte (`0x55`)	Master	Giúp slave đồng bộ baud rate theo timing của master.
PID	1 byte	Master	Gồm 6 bit ID và 2 bit parity để bảo vệ header.
Data	1..8 byte	Master hoặc Slave	Payload thực tế. Demo này dùng kiểu master phát response.
Checksum	1 byte	Bên phát response	Bảo vệ payload, dùng *Enhanced Checksum* trong demo.

2.3 Các loại frame phổ biến trong LIN

Loại frame	Bản chất	Demo này có dùng không
Unconditional frame	Frame cố định theo lịch, slot nào đến thì phát frame đó.	**Có**. Toàn bộ demo đang mô phỏng kiểu này.
Sporadic frame	Master chỉ phát nếu có dữ liệu nào đó đang chờ.	Không.
Event-triggered frame	Master phát header để một slave phù hợp phản hồi.	Không.
Diagnostic frame	Dùng cho chẩn đoán, ví dụ `0x3C` / `0x3D` .	Không ở luồng TX chính, nhưng driver có hỗ trợ sleep/wakeup kiểu diagnostic.

2.4 Lưu ý vật lý: USART không phải là LIN bus hoàn chỉnh

Trong MCU STM32, `USART2` được dùng như một khối phát/thu nối tiếp có hỗ trợ LIN mode để tạo break field và đồng bộ hóa. Tuy nhiên về mặt điện ô tô, bus LIN thật là một đường dây single-wire có mức điện áp và đặc tính vật lý riêng. Nói gọn:

- `USART2` là nơi phát chuỗi byte.
- LIN transceiver là khối chuyển đổi chuỗi byte này thành tín hiệu điện phù hợp với bus LIN.
- Trong log và mô phỏng, ta “nhìn thấy” byte phát từ `USART2` ; đó là cửa sổ quan sát rất gần với bus, nhưng chưa phải toàn bộ câu chuyện vật lý ngoài xe.

---

3. Vì sao AUTOSAR LIN dùng Pull/Time-Triggered

3.1 LIN khác CAN ở triết lý điều phối bus

Khía cạnh	CAN điển hình	LIN Master điển hình
Khi nào frame được phát	Khi upper layer muốn gửi và lower layer chấp nhận	Khi lịch của master cho phép
Ai quyết định thứ tự lên bus	Arbitration trên bus	Master scheduler
Mô hình AUTOSAR thường thấy	*Push* từ COM xuống CanIf/Can	*Pull* từ LinIf lên COM tại đúng slot
Ý nghĩa	Frame có thể phát gần ngay sau khi dữ liệu sẵn sàng	Dữ liệu có thể chờ trong buffer cho tới khi slot đến

> **Điểm rất dễ nhầm:** `Com_SendSignal()` trong demo **không** đẩy dữ liệu ra bus. Hàm này chỉ ghi vào shadow buffer của COM. Bus chỉ hoạt động khi `LinIf_MainFunction()` tới slot và gọi `PduR_LinIfTriggerTransmit()`.

3.2 Kiến trúc Pull thực tế trong repo hiện tại

```
Application
  -> Com_SendSignal(...)          // cập nhật signal mới nhất
  -> LinIf_MainFunction()         // scheduler được gọi định kỳ
      -> PduR_LinIfTriggerTransmit(...)
          -> Com_TriggerTransmit(...)
              -> copy COM shadow buffer sang buffer cục bộ của LinIf
      -> Lin_SendFrame(...)
          -> USART2 phát Break, Sync, PID, Data, Checksum
```

3.3 Nhánh Push có tồn tại nhưng không dùng ở demo này

Trong code vẫn có đường `Com_TriggerIPDUSend() -> PduR_ComTransmit() -> LinIf_Transmit()`. Tuy nhiên file `LinIf.c` cố ý để `LinIf_Transmit()` trả về `E_NOT_OK` cho luồng demo unconditional frame. Mục đích là ép logic theo đúng mô hình pull do scheduler làm chủ.

---

4. Mapping trực tiếp vào code và cấu hình của repo

4.1 Vai trò của từng file

Lớp (Layer)	File thực tế	Nhiệm vụ chính trong demo LIN TX
Application	`example/lin_tx_demo.c`	Khởi tạo hệ thống, cập nhật signal vào COM buffer, gọi `LinIf_MainFunction()` mỗi vòng lặp.
Service	`autosar/com/Com.c`	Chứa *shadow buffer* cho từng I-PDU và pack signal vào đúng byte/bit.
Service	`autosar/pdur/PduR.c`	Route lời gọi `TriggerTransmit` và `TxConfirmation` giữa `LinIf` với `Com`.
ECU Abstraction	`autosar/linif/LinIf.c`	Giữ lịch phát, chọn slot, kéo payload từ COM, tạo `Lin_PduType`, xử lý confirmation.
MCAL	`autosar/lin/Lin.c`	Cấu hình `USART2`, tính PID và checksum, phát break/sync/data ra ngoại vi.

4.2 Mapping từ signal đến frame LIN

Signal	COM I-PDU	Pack trong COM	LinIfTxPduId	Frame ID thô	PID trên wire	I
`ComSig_Light_Headlamp`	`ComConf_ComIPdu_LightCtrl = 3`	`byte0.bit0`	`0`	`0x10`	`0x50`	
`ComSig_Light_DRL`		`byte0.bit1`				
`ComSig_Light_Brightness`		`byte1`				
`ComSig_HVAC_FanSpeed`	`ComConf_ComIPdu_HVACCtrl = 4`	`byte0`	`1`	`0x11`	`0x11`	

4.3 Mapping route trong PduR và lịch trong LinIf

Khối cấu hình	Thông tin quan trọng
`config/PduR_Cfg.c`	`ComConf_ComIPdu_LightCtrl (3) -> LinIfTxPduId 0`, `ComConf_ComIPdu_HVACCtrl (4) -> LinIfTxPduId 1`
`autosar/linif/LinIf.c`	`LinIfTxPduId 0 -> Frame ID 0x10, DLC 4`, `LinIfTxPduId 1 -> Frame ID 0x11, DLC 3`
`LinIf_MainFunction()`	`tick % 10 == 0` phát LightCtrl, `tick % 10 == 5` phát HVACCtrl

> **Lưu ý rất quan trọng về tên trường:** trong `Lin_PduType`, trường có tên `Pid`, nhưng ở demo hiện tại `LinIf` lại gán vào đó **raw frame ID** (`0x10` hoặc `0x11`). Sau đó `Lin_SendFrame()` mới gọi hàm tính parity để biến nó thành **Protected Identifier thật sự** trên wire.

---

5. Luồng thực thi đầy đủ từ `main()` đến `USART2`

5.1 Giai đoạn khởi tạo

Khởi tạo trong demo xảy ra theo đúng thứ tự sau:

```
int main(void)
{
    Log_Init();              // USART1 chỉ dùng log debug
    BSW_Init_LIN_Only();     // LinIf_Init -> PduR_Init -> Com_Init

    while (1) {
        Demo_LIN_LightCtrl();
        Demo_LIN_HVACCtrl();
        LinIf_MainFunction();
        delay_ms(100u);
    }
}
```

- `Log_Init()` cấu hình `USART1` để in log chữ ra terminal.
- `LinIf_Init()` gọi `Lin_Init()`, và chính `Lin_Init()` mới cấu hình `USART2` cho LIN mode.
- `PduR_Init()` bật route table.
- `Com_Init()` xóa toàn bộ shadow buffer về 0.

5.2 Ứng dụng cập nhật signal nhưng chưa gửi bus

Mỗi vòng lặp, ứng dụng cập nhật dữ liệu điều khiển đèn và HVAC qua `Com_SendSignal()`. Hàm này chỉ pack dữ liệu vào đúng byte/bit trong `Com_IpduBuf`:

- `Headlamp` ghi vào `LightCtrl.byte0.bit0`.
- `DRL` ghi vào `LightCtrl.byte0.bit1`.
- `Brightness` ghi vào `LightCtrl.byte1`.
- `FanSpeed` ghi vào `HVACCtrl.byte0`.

> **Trạng thái dữ liệu ngay sau bước này:**
> Dữ liệu đã "mới nhất" ở tầng COM, nhưng bus LIN vẫn hoàn toàn im lặng nếu scheduler của `LinIf` chưa chạm đúng slot.

5.3 Nhánh quyết định trong `LinIf_MainFunction()`

Đây là trái tim của demo. Pseudocode rút gọn nhưng đúng bản chất runtime như sau:

```
if (!s_linIfInited) return;

if (s_txPendingConf != LINIF_INVALID_PDU) {
    st = Lin_GetStatus(0, NULL);
    if (st == LIN_TX_OK) {
        PduR_LinIfTxConfirmation(s_txPendingConf);
        s_txPendingConf = LINIF_INVALID_PDU;
    } else if (st == LIN_TX_ERROR || st == LIN_OK) {
        s_txPendingConf = LINIF_INVALID_PDU;
    }
    return;
}

tick++;
if (tick % 10 == 0) slot = LightCtrl;
else if (tick % 10 == 5) slot = HVACCtrl;
else return;

PduR_LinIfTriggerTransmit(...);
Lin_SendFrame(...);
s_txPendingConf = currentSlot;
```

Ý nghĩa của từng nhánh:

Nhánh	Điều kiện	Ý nghĩa hệ thống
Chưa init	`!s_linIfInited`	Bảo vệ module chưa sẵn sàng.
Pending confirmation	`s_txPendingConf != 255`	Không schedule frame mới, chỉ kiểm tra frame trước đã hoàn tất hay chưa.
Không có slot	`tick % 10` không phải `0` hoặc `5`	Không phát gì trong vòng này.
Slot HVAC	`tick % 10 == 5`	Kéo payload HVAC từ COM và phát frame ID `0x11`.
Slot Light	`tick % 10 == 0`	Kéo payload Light từ COM và phát frame ID `0x10`.

5.4 Pha kéo dữ liệu: `LinIf -> PduR -> Com`

Khi trúng slot, `LinIf` không lấy dữ liệu trực tiếp từ ứng dụng mà đi qua tuyến chuẩn AUTOSAR:

```
PduInfoType pduInfo;
pduInfo.SduDataPtr = s_local_tx_buffer;
pduInfo.SduLength  = configuredDlc;

PduR_LinIfTriggerTransmit(LinIfTxPduId, &pduInfo);
    -> Com_TriggerTransmit(ComTxPduId, &pduInfo);
    -> memcpy(s_local_tx_buffer, Com_IpduBuf[ComTxPduId], len);
```

Kết quả sau bước này:

- `Com_IpduBuf` vẫn giữ vai trò *source of truth* ở tầng COM.
- `s_local_tx_buffer` của `LinIf` giữ snapshot payload dùng cho đúng lần phát hiện tại.
- Payload đã tách rời khỏi ứng dụng; từ đây trở xuống chỉ còn việc đóng gói và phát ra bus.

5.5 Pha đóng gói ở `LinIf`

Sau khi kéo được payload, `LinIf` tạo cấu trúc `Lin_PduType`:

```
linPdu.Pid    = 0x10 hoặc 0x11;     // raw frame ID trong demo
linPdu.Cs     = LIN_ENHANCED_CS;
linPdu.Drc    = LIN_FRAMERESPONSE_TX;
linPdu.Dl     = pduInfo.SduLength;
linPdu.SduPtr = pduInfo.SduDataPtr;

Lin_SendFrame(0, &linPdu);
```

Ba điểm cần nhớ:

- `Drc = LIN_FRAMERESPONSE_TX` nghĩa là master phát cả header lẫn response data.
- `Cs = LIN_ENHANCED_CS` nghĩa là checksum tính trên **PID + Data**.
- `Pid` ở đây mới là raw ID, chưa phải byte header cuối cùng trên wire.

5.6 Pha driver ở `Lin.c`: từ cấu trúc phần mềm sang chuỗi byte thật

Khi vào `Lin_SendFrame()`, driver làm lần lượt:

1. Kiểm tra module đã init và `Dl` nằm trong khoảng `1..8`.
2. Đặt `s_linStatus = LIN_TX_BUSY`.
3. Gửi **Break** bằng `USART_SendBreak(USART2)`.
4. Gửi **Sync** `0x55`.
5. Tính **PID** từ raw frame ID bằng parity LIN.
6. Gửi từng byte data.
7. Tính và gửi **Checksum**.
8. Đặt `s_linStatus = LIN_TX_OK`.

> **Sửa hiểu nhầm thường gặp:** phần phát LIN ở demo dùng `USART2`, không phải `USART1`. `USART1` chỉ phục vụ log debug qua `Log_Print()`.

5.7 Pha xác nhận truyền ở vòng lặp kế tiếp

Sau khi gọi `Lin_SendFrame()` thành công, `LinIf` đặt `s_txPendingConf`. Ở vòng lặp sau, trước khi xét slot mới, nó đi vào nhánh kiểm tra trạng thái:

```
st = Lin_GetStatus(0, NULL);
if (st == LIN_TX_OK) {
    PduR_LinIfTxConfirmation(s_txPendingConf);
    Com_TxConfirmation(mappedComPduId);
}
```

Trong demo hiện tại, driver đang truyền theo kiểu blocking và đặt `LIN_TX_OK` ngay trong lần gọi phát. Vì vậy vòng tiếp theo gần như luôn chỉ làm nhiệm vụ xác nhận rồi `return`. Đây là một mô phỏng "semi-blocking nhưng vẫn giữ shape kiến trúc AUTOSAR".

---

6. Phân tích LIN frame ở mức byte/bit

6.1 Công thức Protected Identifier (PID)

PID gồm 6 bit ID và 2 bit parity:

```
P0 = ID0 xor ID1 xor ID2 xor ID4
P1 = not(ID1 xor ID3 xor ID4 xor ID5)
PID = ID[5:0] | (P0 << 6) | (P1 << 7)
```

Frame ID thô	Giải thích	PID trên wire
`0x10`	Parity cho ra `P0=1`, `P1=0`	`0x50`
`0x11`	Parity cho ra `P0=0`, `P1=0`	`0x11`

6.2 Công thức checksum trong demo

Demo dùng **Enhanced Checksum**, tức là checksum tính trên:

```
sum = PID + Data[0] + Data[1] + ... + Data[n-1]
nếu sum >= 256 thì sum = sum - 255 cho mỗi lần carry
checksum = bitwise_not(sum) & 0xFF
```

6.3 Giải mã hai frame đầu tiên trong log thật

Trích từ log runtime, hai frame đầu tiên nhìn thấy là:

```
[UART2] [Break] [0x55] [0x11] [0x55] [0x00] [0x00] [0x99]
[UART2] [Break] [0x55] [0x50] [0x02] [0x09] [0x00] [0x00] [0xA4]
```

Frame	Diễn giải
HVAC đầu tiên	`Break` mở đầu frame, `0x55` là Sync, `0x11` là PID. Data = `0x55 0x00 0x00`. Ở thời điểm này `fanSpeed = 0x55`. Checksum: `0x11 + 0x55 + 0x00 + 0x00 = 0x66` nên `~0x66 = 0x99`.
Light đầu tiên	`0x50` là PID của raw frame ID `0x10`. Data = `0x02 0x09 0x00 0x00` nghĩa là: `byte0 = 0x02` - Headlamp = 0, DRL = 1; `byte1 = 0x09` - Brightness = 9. Checksum: `0x50 + 0x02 + 0x09 + 0x00 + 0x00 = 0x5B` nên `~0x5B = 0xA4`.

6.4 Vì sao frame đèn đầu tiên lại có brightness = `0x09`, không phải `0x64`?

Đây là chi tiết rất hay để hiểu đúng runtime. Ứng dụng cập nhật brightness ở **mọi vòng lặp**, nhưng `tick` trong `LinIf` chỉ tăng ở những vòng không bị nhánh confirmation chặn lại. Do sau mỗi lần gửi frame sẽ có một vòng chuyên xử lý confirmation, thời điểm chạm slot `tick = 10` thực tế xảy ra muộn hơn số vòng ứng dụng cập nhật dữ liệu. Kết quả là brightness đã chạy qua `100` rồi vòng tiếp theo thành `9`, và giá trị này mới là snapshot được kéo xuống bus.

---

7. Timeline thực tế của những vòng lặp đầu tiên

Bảng dưới đây mô tả các vòng lặp đầu tiên để thấy rõ tương quan giữa **application update**, **tick** của scheduler và **frame nào thực sự được phát**:

Vòng lặp	Giá trị ứng dụng sau khi update	Nhánh trong `LinIf_MainFunction()`	Kết quả
1..4	Brightness tăng dần, FanSpeed tăng dần	`tick = 1..4`, chưa trúng slot	Không có frame nào lên bus
5	`fanSpeed = 0x55`	`tick = 5`, trúng slot HVAC	Phát frame HVAC: `[Break] [0x55][0x11][0x55][0x00] [0x00][0x99]`
6	Ứng dụng vẫn cập nhật tiếp	Nhánh `pending confirmation`	Không tăng tick, chỉ gọi callback xác nhận
7..10	Brightness tiếp tục đổi, Headlamp/DRL tiếp tục toggle	`tick = 6..9`	Chưa có frame mới
11	`headlamp = 0`, `drl = 1`, `brightness = 0x09`	`tick = 10`, trúng slot Light	Phát frame Light: `[Break] [0x55][0x50][0x02][0x09] [0x00][0x00][0xA4]`

> **Hệ quả quan trọng:** lịch trong demo là lịch "logic" theo `tick`, nhưng vì nhánh confirmation chiếm riêng một vòng lặp nên khoảng thời gian thực tế giữa hai lần phát frame trên tường đồng hồ dài hơn số nhân đơn giản của `delay_ms(100u)`.

---

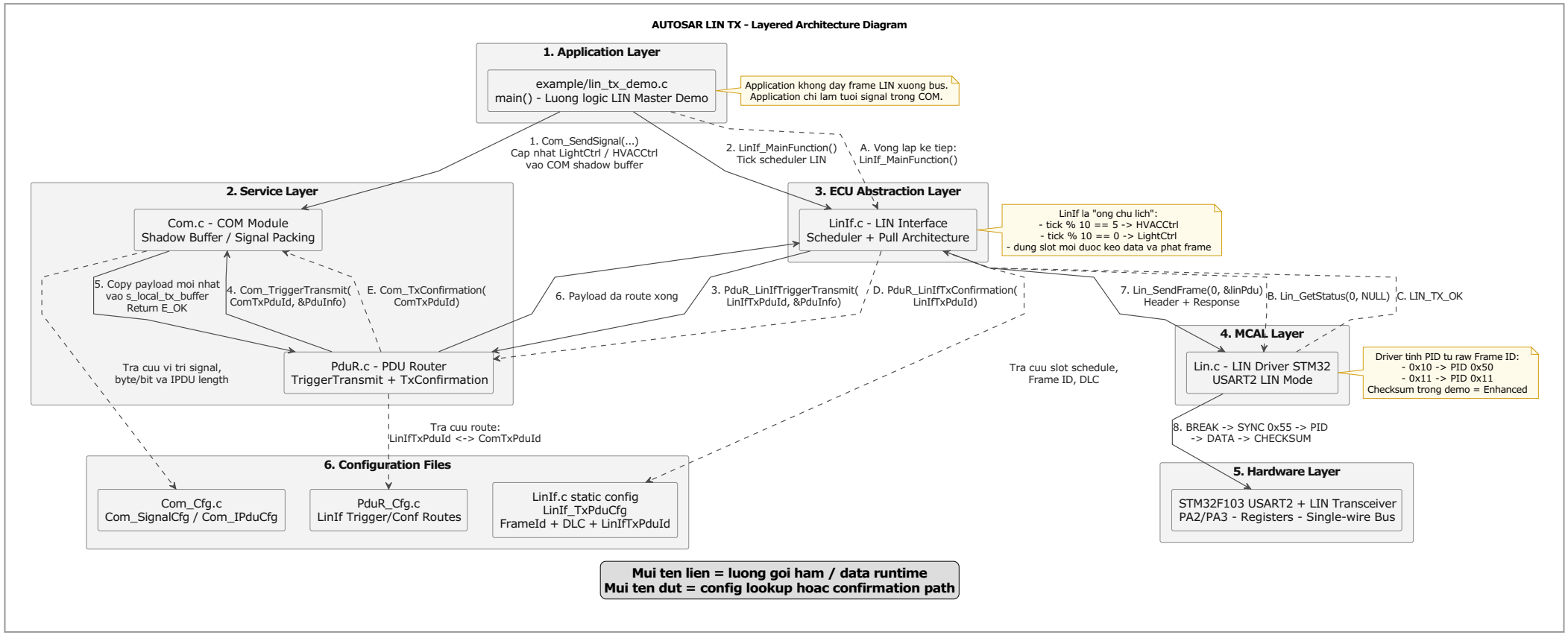
8. Sơ đồ chi tiết đã render sẵn

Để tài liệu chuyển sang PDF đẹp và ổn định, phần sơ đồ được tổ chức theo cơ chế **mỗi sơ đồ một sheet riêng**. Cách này giảm nguy cơ một hình bị chẻ đôi giữa hai trang khi dùng công cụ HTML -> PDF trên PC.

> **Nguyên tắc dàn trang áp dụng cho PDF:** sơ đồ tổng quan và sơ đồ nhánh chính được đặt vào từng trang riêng, có khung riêng, có caption riêng, và toàn bộ block bị đánh dấu `page-break-inside: avoid`. Sơ đồ sequence cực dài được chuyển sang phụ lục điện tử để tránh việc thu nhỏ quá mức trên một trang A4 rồi khó đọc.

Hình 1. Sơ đồ kiến trúc lớp kiểu overview

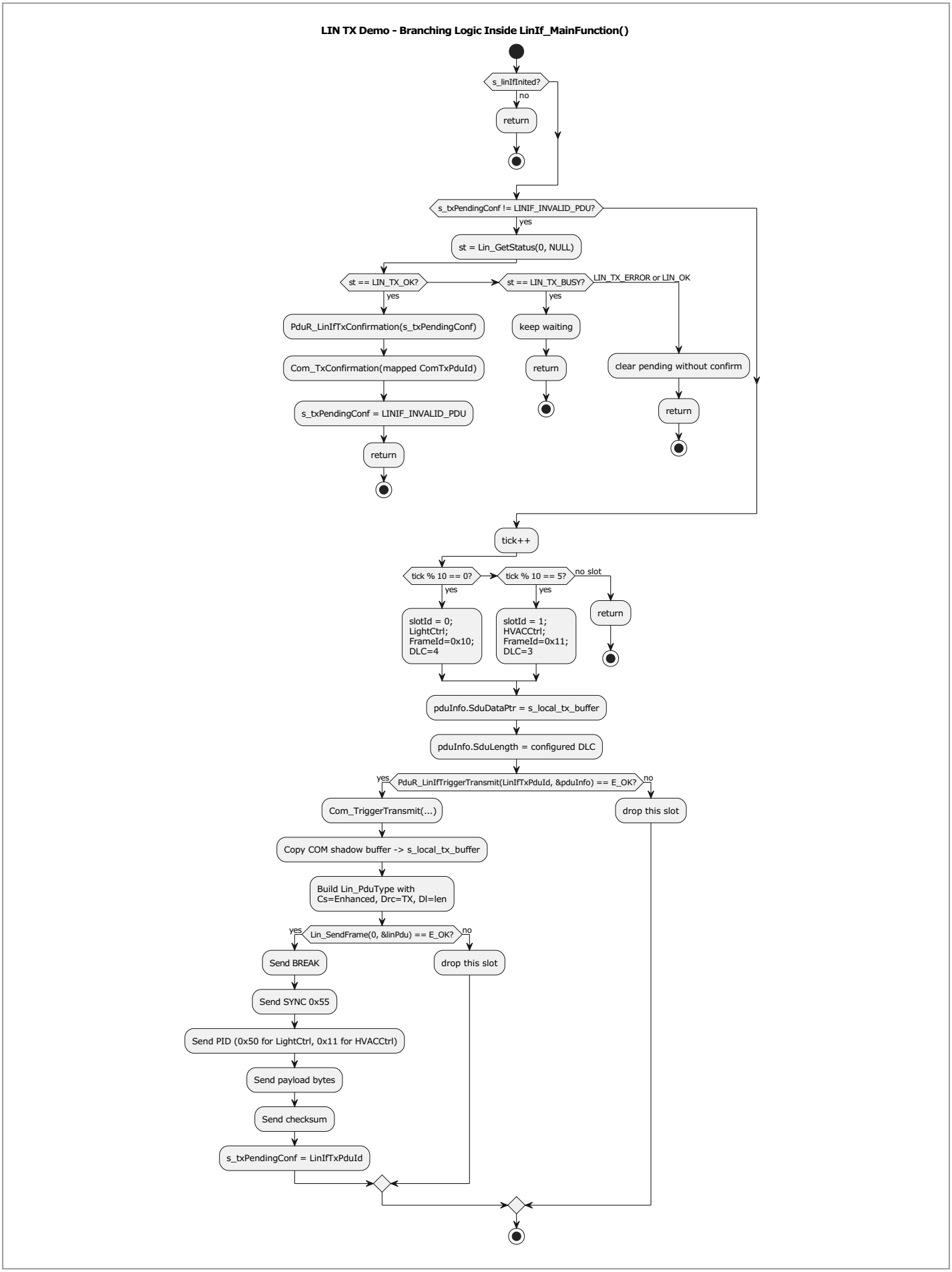
Phù hợp để đưa vào PDF vì thể hiện được toàn bộ lớp AUTOSAR, tuyến runtime chính, lookup cấu hình và nhánh confirmation trong một hình duy nhất.



Luồng chính: Application -> COM -> PduR -> LinIf -> Lin -> USART2/LIN bus. Các nét đứt mô tả lookup cấu hình và callback xác nhận truyền.

Hình 2. Sơ đồ logic các nhánh trong `LinIf_MainFunction()`

Sơ đồ này nhấn vào scheduler, nhánh pending confirmation, slot selection và điều kiện kéo dữ liệu/phát frame.



Mỗi lần gọi `LinIf_MainFunction()` hoặc xử lý xác nhận khung cũ, hoặc tăng `tick` để kiểm tra slot mới, rồi mới quyết định có được phép kéo payload từ COM hay không.

Phụ lục điện tử: sơ đồ sequence đầy đủ

Sơ đồ sequence chi tiết từ `main()` đến từng byte trên `USART2` vẫn được giữ lại, nhưng không nhúng trực tiếp vào thân PDF chính vì chiều cao quá lớn. Nếu ép vừa một trang A4, chữ sẽ nhỏ và kém đọc; nếu không ép, nhiều tool sẽ cắt sơ đồ qua hai trang.

Các file nguồn và bản render đầy đủ:

- `lin_tx_demo_full_flow.puml` - nguồn PlantUML đầy đủ.
- `lin_tx_demo_full_flow.svg` - sequence diagram chi tiết.
- `lin_tx_demo_full_flow_001.svg` - activity diagram chi tiết.
- `lin_layered_architecture_like_sample.puml` - sơ đồ overview kiểu layered như bản mẫu.

Khuyến nghị thực tế: bản PDF chính dùng Hình 1 và Hình 2; còn khi cần soi sâu call chain/byte-level thì mở thêm file SVG sequence trên máy tính để phóng to.

---

9. Kiểm chứng bằng log và lab mô phỏng

Có thể kiểm chứng nhanh hành vi của demo bằng Renode hoặc bằng file log đã thu sẵn.

9.1 Điểm cần quan sát trong log

- Dòng `[LinIf] PduR returned E_OK!` cho biết pha Pull từ `Com` đã thành công.
- Dòng `[UART2] [Break] ...` cho thấy chuỗi byte thật sự đã đi vào ngoại vi LIN.
- Dòng `Lin_SendFrame SUCCESS!` cho biết driver đã hoàn thành việc phát frame.

9.2 Trích log minh họa

```
--- Node TX (LIN Only Demo) Booted ---
[LinIf] PduR returned E_OK!

--- LIN MCAL Gửi Khung Mới ---
[UART2] [Break] [0x55] [0x11] [0x55] [0x00] [0x00] [0x99] [LinIf] Lin_SendFrame SUCCESS!
0x55 0x00 0x00
[LinIf] PduR returned E_OK!

--- LIN MCAL Gửi Khung Mới ---
[UART2] [Break] [0x55] [0x50] [0x02] [0x09] [0x00] [0x00] [0xA4] [LinIf] Lin_SendFrame SUCCESS!
0x02 0x09 0x00 0x00
```

9.3 Nếu muốn quan sát sâu hơn trong lab

- Đặt breakpoint tại `LinIf_MainFunction` để theo dõi biến `tick` và `s_txPendingConf`.
- Đặt breakpoint tại `Com_TriggerTransmit` để xem buffer nào đang bị copy.
- Đặt breakpoint tại `Lin_SendFrame` để xem raw frame ID, DLC và địa chỉ payload.
- Quan sát log `USART2` để xác thực PID và checksum có đúng với công thức LIN không.

---

10. Các điểm dễ nhầm và kết luận kỹ thuật

10.1 Những điểm dễ nhầm nhất

- **`Com_SendSignal()` không gửi frame.** Nó chỉ pack signal vào shadow buffer.
- **`LinIf_Transmit()` không phải đường chính của demo.** Demo unconditional frame đang dùng Pull qua `PduR_LinIfTriggerTransmit()`.
- **`Lin_PduType.Pid` trong demo không phải PID cuối cùng trên wire.** Đó là raw frame ID trước khi driver thêm parity.
- **`USART1` không phải ngoại vi LIN.** `USART1` chỉ in log; `USART2` mới phát chuỗi byte LIN.
- **Confirmation bị tách sang vòng sau.** Vì vậy timeline thực tế của demo không đơn giản là "mỗi 5 hoặc 10 vòng là phát ngay" nếu nhìn theo thời gian tuyệt đối.

10.2 Kết luận cuối cùng

Điều quan trọng nhất để hiểu LIN trong AUTOSAR Classic là: **master không bị dữ liệu kéo đi, mà chính master scheduler quyết định khi nào dữ liệu được quyền đi xuống bus**. Trong demo này:

- Ứng dụng liên tục làm tươi trạng thái hệ thống.
- `Com` giữ ảnh chụp mới nhất của từng I-PDU.
- `LinIf` là người canh lịch và chủ động kéo dữ liệu đúng thời điểm.
- `PduR` làm nhiệm vụ nối đúng route giữa các tầng.
- `Lin` biến cấu trúc phần mềm thành chuỗi byte vật lý theo chuẩn LIN.

Nắm được chuỗi này thì việc đọc tiếp các chủ đề nâng cao như sleep/wakeup, diagnostic frame, event-triggered frame hay tích hợp OS task thật sẽ dễ hơn rất nhiều.